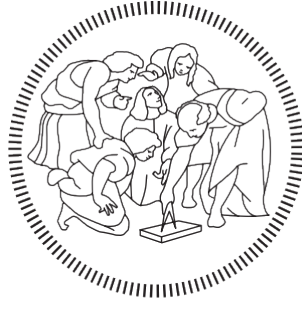




POLITECNICO

MILANO 1863

Prova Finale di Reti Logiche
Anno Accademico 2023-2024

Alessandro Marzio Carrieri 10754069, Federica Currò Dossi 10811197

Indice

1	Introduzione	3
1.1	Specifica	3
1.2	Componente	3
2	Architettura	4
2.1	Modulo Register Value	5
2.2	Modulo Register Credibility	5
2.3	Modulo Register Output	6
2.4	Modulo Register Address	6
2.5	Modulo Counter	7
2.6	Modulo K-Comparator	7
2.7	FSM	8
2.7.1	Segnali di Input	8
2.7.2	Segnali Output di default	9
2.7.3	Stati FSM	10
3	Risultati sperimentali	12
3.1	Sintesi	12
3.2	Simulazioni	13
4	Conclusioni	13

1 Introduzione

Il suddetto documento tratta degli approcci progettuali implementati per lo sviluppo della prova finale di reti logiche. Si introduce la specifica assegnataci sulla quale viene fatta un'analisi del funzionamento e delle criticità riscontrate nella progettazione della soluzione, esplicitando attraverso l'architettura da noi proposta una scelta risolutiva del problema.

1.1 Specifica

Il progetto assegnatoci prevede lo sviluppo di un modulo hardware che legge una sequenza di parole dalla memoria. Il modulo riceve in ingresso un input seriale di i_k parole il cui valore è compreso tra 0 e 255, ove lo 0 assume un valore non specificato, e un indirizzo di memoria i_add che punta alla prima parola della sequenza. La sequenza di parole è memorizzata a partire da i_add ogni 2 byte. Il modulo deve completare la sequenza sostituendo gli 0 con l'ultimo valore non nullo letto e scrivendo un valore di credibilità nel byte successivo a ogni parola. Il valore della credibilità è inizializzato a 31 e deve essere decrementato ogni volta che viene letto uno 0 nella sequenza delle parole, senza mai risultare negativa.

1.2 Componente

L'interfaccia del componente da implementare è la seguente:

```
entity project_reti_logiche is
    port (

        i_clk   : in std_logic;
        i_rst   : in std_logic;
        i_start : in std_logic;
        i_add   : in std_logic_vector(15 downto 0);
        i_k     : in std_logic_vector(9  downto 0);
        i_mem_data: in std_logic_vector(7  downto 0);

        o_done : out std_logic;

        o_mem_en  : out std_logic;
        o_mem_we  : out std_logic;
        o_mem_addr: out std_logic_vector(15 downto 0);
        o_mem_data: out std_logic_vector(7  downto 0)
    );
end entity;
```

Segnali di Input:

- i_clk : segnale di clock.
- i_rst : segnale di reset asincrono che reinizializza il modulo.
- i_start : segnale sincrono di inizio operazione.
- i_add : segnale sincrono che contiene l'indirizzo della prima parola della sequenza.
- i_k : segnale sincrono che specifica il numero di parole della sequenza.
- i_mem_data : segnale che contiene il valore della parola letto dalla memoria.

Segnali di Output:

- o_done : segnale di fine elaborazione del modulo.
- o_mem_en : segnale per abilitare la comunicazione con la memoria, sia in lettura che in scrittura.
- o_mem_we : segnale per abilitare la scrittura in memoria. In caso di lettura è necessario sia 0.

- o_mem_addr : segnale contenente l'indirizzo di memoria dal quale si voglia scrivere o leggere un valore.
- o_mem_data : segnale che contiene il valore da scrivere in memoria.

Esempio di funzionamento. Le parole sono riportate in grassetto.

Sequenza iniziale: **0 0 13 0 0 0 14 0 123 0 0 0 0 0 0**

Sequenza finale: **0 0 13 31 13 30 14 31 123 31 123 30 123 29 123 28**

2 Architettura

Al fine di sviluppare un' architettura in grado di seguire un flusso coerente con le specifiche assegnateci, abbiamo privilegiato un'implementazione nella quale ogni componente avesse una o più funzionalità specifiche, orchestrate da segnali di ingresso. L'architettura è stata divisa in sette moduli, dei quali uno è rappresentato da una macchina a stati finiti, il cui funzionamento è sincrono rispetto al segnale di clock in ingresso; la scelta di renderli sincroni permette un controllo assoluto sui componenti, di conseguenza l'eventuale presenza di un errore e/o comportamento ambiguo può essere rilevata facilmente.

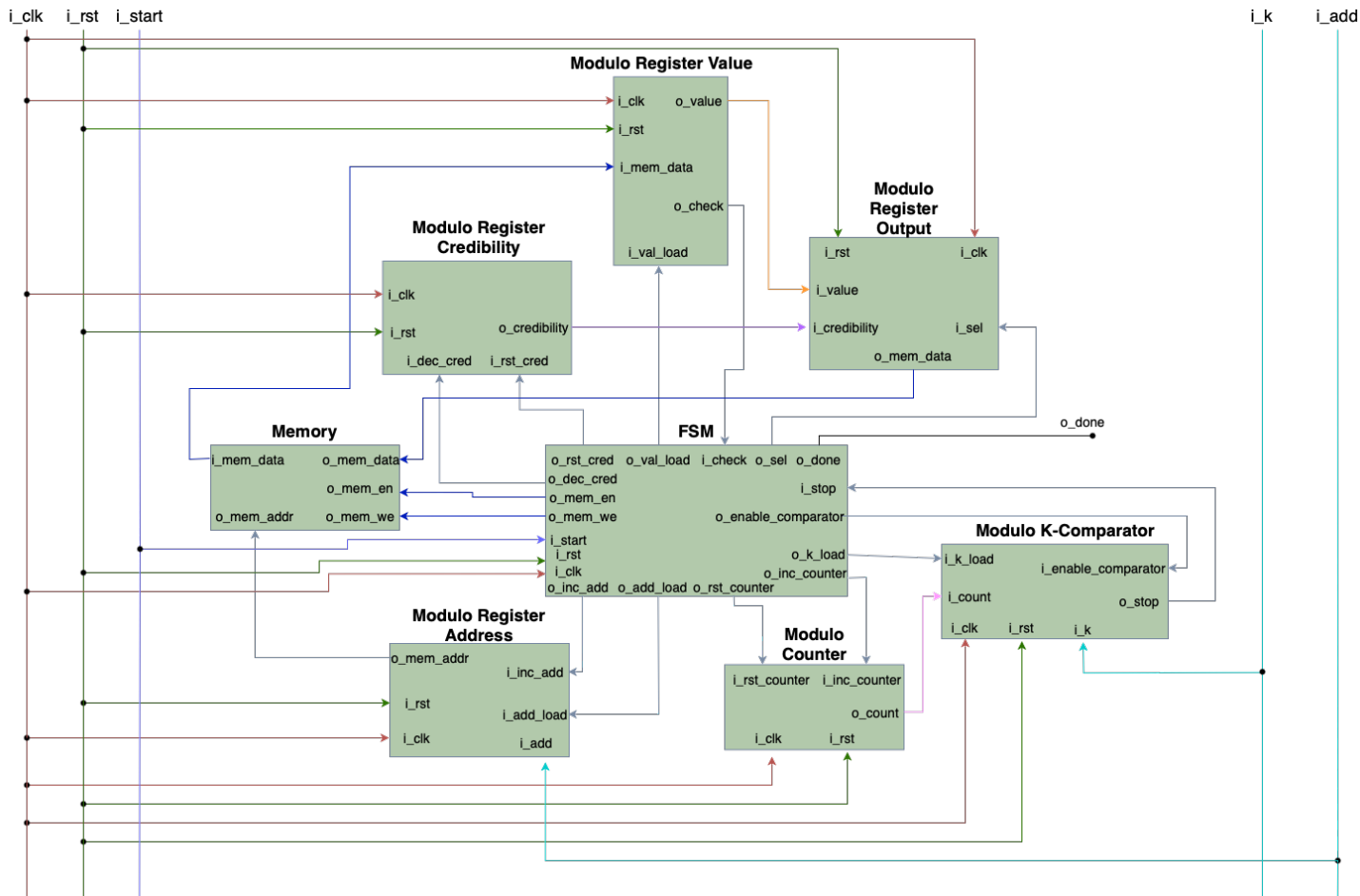


Figura 1: Architettura completa.

2.1 Modulo Register Value

Modulo utilizzato per la lettura, salvataggio e valutazione del valore presente in *i_mem_data*. La lettura del valore in ingresso è regolata da *i_val_load*, segnale proveniente dalla FSM. Attraverso un processo sequenziale sincronizzato sul fronte di salita di *i_clk* quando *i_val_load* è alto, il componente legge il valore contenuto in *i_mem_data* e lo confronta con zero, se il confronto risulta positivo il segnale di *o_check* viene posto a zero, il valore in ingresso non verrà salvato, viceversa se quest'ultimo è diverso da zero, *o_check* viene posto a uno e il valore in ingresso viene salvato nel registro. Il segnale *o_check* di output è collegato in input alla FSM, mentre il segnale di *o_value* (corrispondente al valore salvato in precedenza) è collegato in input al Modulo Register Output.

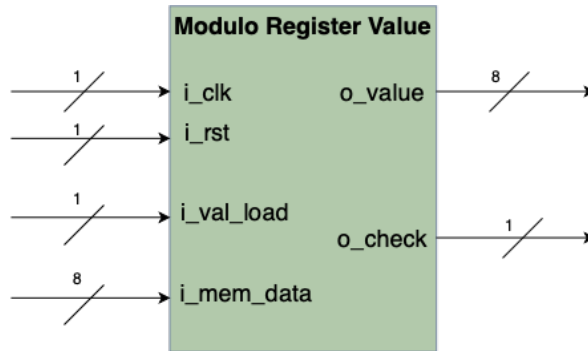


Figura 2: Modulo Register Value.

2.2 Modulo Register Credibility

Modulo utilizzato per la corretta gestione del valore di credibilità, implementata attraverso l'utilizzo di un registro per memorizzare il valore corretto. Il registro interno è inizializzato a 31, scelta conseguita in seguito alla valutazione della specifica. La gestione del valore è regolata da due segnali di input provenienti dalla FSM nel seguente modo:

- Se *i_dec_cred* è uguale a 1 il valore memorizzato nel registro viene decrementato. Prima del decremento il valore viene confrontato con zero e in caso di confronto positivo non sarà decrementato ulteriormente, ma riportato nullo, come richiesto dalla specifica.
- Se *i_rst_cred* è uguale a 1 il valore presente nel registro viene posto nuovamente a 31.

In tutti gli altri casi, il valore salvato sarà riportato in uscita senza cambiamenti.

L'assegnamento a *o_credibility* del valore memorizzato avviene mediante un processo combinatorio ed è collegato in input al Modulo Register Output. I controlli e operazioni (decremento e reset) sopracitati sono svolti in un processo sequenziale sincronizzato sul fronte di salita di *i_clk*. La scelta di utilizzare un segnale di reset aggiuntivo (*i_rst_cred*) ci ha inoltre permesso di gestire la lettura di più sequenze di parole consecutive, mentre in presenza di più reset asincroni il segnale *i_rst* permette di reinizializzare il registro a 31.

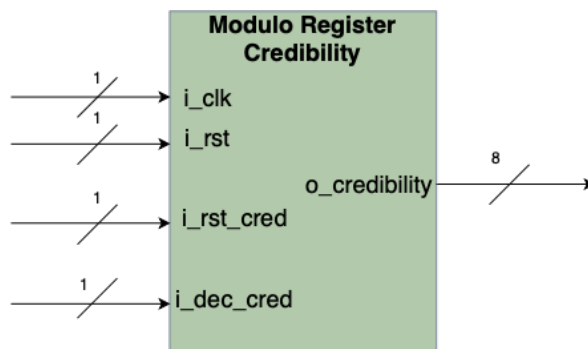


Figura 3: Modulo Register Credibility.

2.3 Modulo Register Output

Il Modulo Register Output si occupa della corretta gestione del valore da scrivere in memoria. Il segnale *i_value* proviene dal Modulo Register Value, mentre *i_credibility* dal Modulo Register Credibility. La scelta di quale dei due valori di input salvare nel registro e successivamente scrivere in memoria è gestita da *i_sel*, proveniente dalla FSM.

Quando *i_sel* è 0, viene salvato *i_value*, in caso contrario verrà salvato *i_credibility*.

Il salvataggio è coordinato da un processo sequenziale sincronizzato sul fronte di salita di *i_clk* e in presenza di un reset asincrono il registro viene inizializzato a zero, mentre il processo di assegnamento dell'output *o_mem_data* è combinatorio.

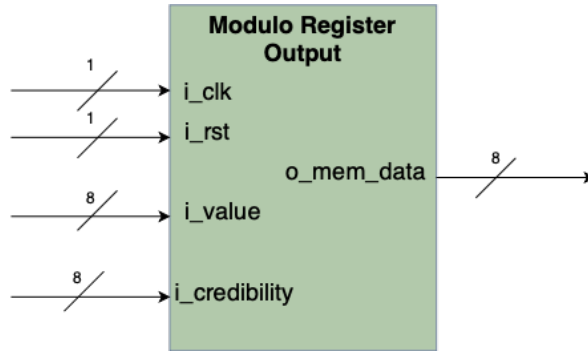


Figura 4: Modulo Register Output.

2.4 Modulo Register Address

Il Modulo Register Address si occupa di salvare il valore del primo indirizzo di memoria *i_add* e di incrementarlo. L'incremento è gestito dal segnale *i_inc_add* e il salvataggio da *i_add_load*, entrambi provenienti dalla FSM. Il funzionamento del modulo è diviso in due processi, uno sequenziale sincrono sul fronte di salita di *i_clk*, in cui l'indirizzo viene salvato o incrementato e uno combinatorio in cui avviene il corretto assegnamento dell'indirizzo da inviare alla memoria. In caso di reset asincrono il valore nel registro viene inizializzato a zero.

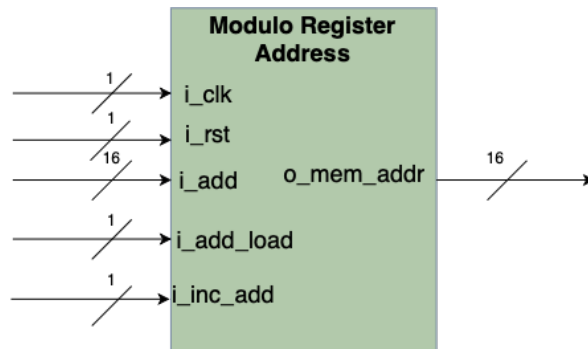


Figura 5: Modulo Register Address.

2.5 Modulo Counter

Il Modulo Counter implementa un ciclo di conteggio, coordinato dal segnale in ingresso *i_inc_counter* che se posto a uno incrementa il contatore di un'unità e dal segnale *i_rst_counter* che mi riporta il contatore a zero, il tutto attraverso un processo sincrono sul fronte di salita di *i_clk*. Si precisa che i segnali di reset del conteggio e incremento provengono dalla FSM. Il risultato del conteggio viene posto su *o_count* mediante un processo combinatorio e letto dal Modulo K-Comparator. In presenza di un reset asincrono il registro interno viene inizializzato a zero.

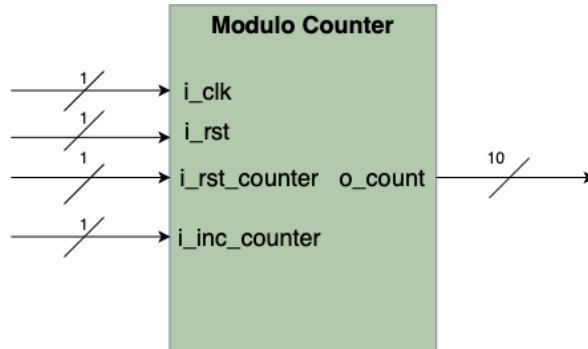


Figura 6: Modulo Counter.

2.6 Modulo K-Comparator

Tale modulo si occupa del salvataggio del valore di *i_k* e del confronto di esso con il valore *i_count* proveniente dal Modulo Counter. Se i due valori coincidono il segnale di *o_stop* collegato in input alla FSM viene posto a 1, in caso contrario è posto al suo valore di default 0. Il salvataggio di *i_k* è abilitato attraverso il segnale *i_k_load* e il confronto mediante *i_enable_comparator* entrambi provenienti dalla FSM. In presenza di reset asincrono il contenuto del registro interno viene inizializzato a zero, in modo da poter gestire il caso in cui *i_k* sia nullo.

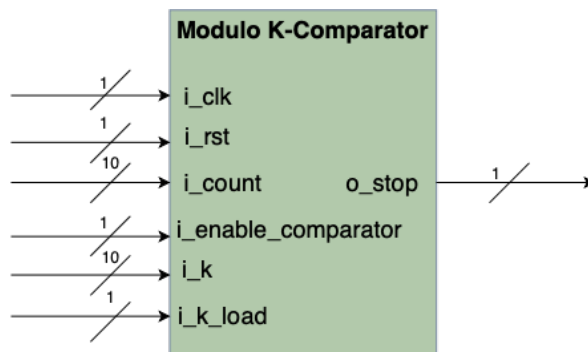


Figura 7: Modulo K-Comparator.

2.7 FSM

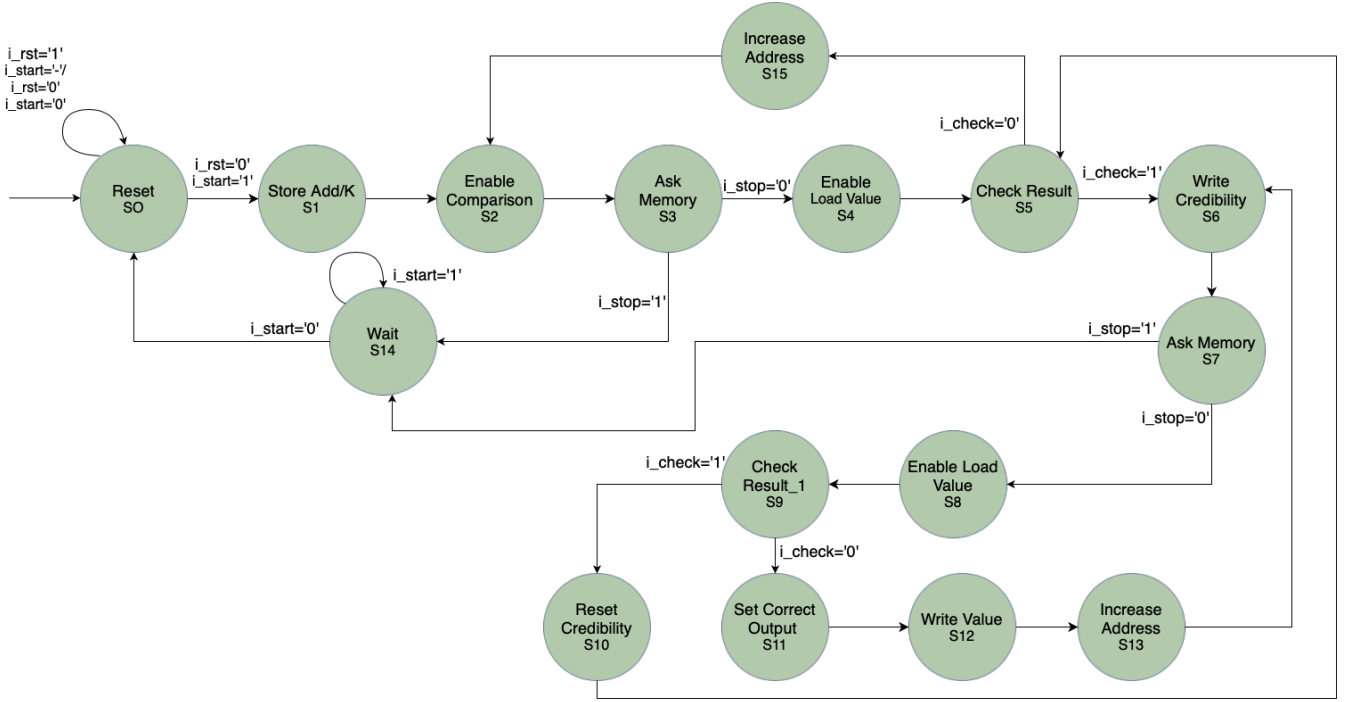


Figura 8: Finite State Machine

Il modulo FSM contiene una macchina a stati finiti di Moore, in cui l'output non dipende dall'input letto, ma solamente dallo stato presente. La FSM controlla il flusso dell'architettura completa, orchestrando attraverso i suoi segnali di output il salvataggio di valori, la scrittura in memoria e l'interazione con essa.

L'architettura del componente è sviluppata mediante due processi:

- **Current State Process:** processo sequenziale sincronizzato sul fronte di salita del clock che determina lo stato corrente della FSM.
- **Output Process:** processo combinatorio che ha nella sua sensitivity list il segnale interno *curr_state* indicante lo stato corrente. In tale processo vengono assegnati ai segnali in output i valori corretti.

Si precisa che in presenza di un segnale *i_rst* alto, lo stato corrente sarà sempre Reset (S0), ciò non è stato riportato in figura al fine di rendere la rappresentazione più chiara.

2.7.1 Segnali di Input

Input Signal	#bit	Note
<i>i_clk</i>	1	Segnale di clock.
<i>i_rst</i>	1	Segnale di reset per riportare lo stato corrente a S0.
<i>i_start</i>	1	Segnale di inizio elaborazione.
<i>i_stop</i>	1	Segnale di fine elaborazione proveniente dal Modulo K-Comparator. Se alto indica che ho letto <i>i_k</i> parole.
<i>i_check</i>	1	Segnale proveniente dal Modulo Register Value, se alto indica che la parola letta è diversa da zero, viceversa se è basso.

Tabella 1: Segnali connessi in input alla FSM.

2.7.2 Segnali Output di default

Di seguito si riportano i valori dei segnali di Output di default e il Modulo dell'architettura ai quali sono collegati.

Output Signal	#bit	Value	Note
<i>o_val_load</i>	1	0	Il suo valore 0 indica che il Modulo Register Value non è abilitato alla lettura dell'input <i>i_mem_data</i> .

Tabella 2: Segnali connessi in input al Modulo Register Value.

Output Signal	#bit	Value	Note
<i>o_rst_cred</i>	1	0	Verrà posto a 1 quando sarà necessario resettare il valore di credibilità a 31 durante il flusso di funzionamento.
<i>o_dec_cred</i>	1	0	Verrà posto a 1 quando sarà necessario decrementare il valore di credibilità durante il flusso di funzionamento.

Tabella 3: Segnali connessi in input al Modulo Register Credibility.

Output Signal	#bit	Value	Note
<i>o_sel</i>	1	1	Il valore 1 indica che nel Modulo verrà salvato e in seguito scritto in memoria il valore di credibilità.

Tabella 4: Segnali connessi in input al Modulo Register Output.

Output Signal	#bit	Value	Note
<i>o_add_load</i>	1	0	Il salvataggio del primo indirizzo di memoria non è abilitato.
<i>o_inc_add</i>	1	0	L'incremento del valore di indirizzo presente nel Modulo non è abilitato.

Tabella 5: Segnali connessi in input al Modulo Register Address.

Output Signal	#bit	Value	Note
<i>o_rst_counter</i>	1	0	Il reset del contatore non è abilitato, sarà abilitato quando sarà necessario leggere una seconda sequenza di parole in assenza di una reinizializzazione del modulo.
<i>o_inc_counter</i>	1	0	L'incremento del contatore non è abilitato, sarà abilitato quando verrà letta una parola dalla sequenza in memoria.

Tabella 6: Segnali connessi in input al Modulo Counter.

Output Signal	#bit	Value	Note
<i>o_enable_comparator</i>	1	0	Il confronto tra il valore di <i>i_k</i> e il valore del conteggio proveniente dal Modulo Counter non è abilitato. Questo segnale sarà posto a 1 prima della lettura in memoria di una parola.
<i>o_k_load</i>	1	0	Il salvataggio di <i>i_k</i> non è abilitato.

Tabella 7: Segnali connessi in input al Modulo K-Comparator.

2.7.3 Stati FSM

Di seguito si riporta il funzionamento di ciascun stato della FSM, ove non specificato un determinato segnale di output si precisa che il suo valore corrisponde al valore di default riportato in 2.7.2.

- Reset (S0): Stato iniziale di reset in cui si attende che i_rst sia uguale a 0 e i_start sia uguale a 1. Come riportato nel disegno di 2.7 la presenza di un segnale di i_rst posto a 1 mi permetterà di rimanere in S0 qualsiasi sia il valore di i_start (riportato come Don't Care), e si rimarrà in S0 anche in presenza di un segnale di i_rst uguale a 0 in concomitanza con un segnale di i_start uguale a 0. Si passerà allo stato Enable Comparison (S1) solo quando i_rst sarà uguale a 0 e i_start uguale a 1.
- Store Add/K (S1): Stato per abilitare il salvataggio nei registri dedicati dei valori di i_add e i_k .

Output Signal	Value	Note
o_add_load	1	Il salvataggio di i_add è abilitato.
o_k_load	1	Il salvataggio di i_k è abilitato.

Tabella 8: Segnali di output in S1.

- Enable Comparison (S2): Stato che abilita il confronto tra i_k e il valore di i_count tale confronto verrà svolto dal Modulo K-Comparator, come già specificato in 2.6. La presenza di questo stato a inizio computazione permette di gestire il caso in cui i_k sia nullo.

Output Signal	Value	Note
$o_enable_comparator$	1	Il confronto tra il valore di parole della sequenza i_k e il numero di parole già lette i_count è abilitato.

Tabella 9: Segnali di Output in S2.

- Ask Memory (S3): Stato che controlla l'input i_stop proveniente dal Modulo K-Comparator, il quale indica l'esito del confronto abilitato in S2. Se ho letto i_k parole allora lo stato corrente diventerà Wait (S14), in caso contrario sarà Enable Load Value (S4). I valori di output non cambiano in tale stato a seconda del valore di i_stop , poichè un eventuale valore positivo del segnale di stop non comporterà comunque una scrittura di un valore in memoria.

Output Signal	Value	Note
o_mem_en	1	Viene abilitata l'interazione con la memoria.
$o_inc_counter$	1	Viene incrementato il numero di parole lette.

Tabella 10: Segnali di Output in S3.

- Enable Load Value (S4): stato in cui viene abilitata la lettura del valore i_mem_data nel Modulo Register Value. Come già descritto in sezione 2.1 viene svolto il controllo di nullità su i_mem_data .

Output Signal	Value	Note
o_val_load	1	Viene abilitata la lettura del valore i_mem_data .

Tabella 11: Segnali di Output in S4.

- Check Result (S5): stato in cui viene controllato i_check proveniente dal Modulo Register Value, frutto di elaborazione specificata in 2.1. In caso di i_check uguale a 1 lo stato successivo sarà Write Credibility (S6), nel caso contrario sarà Increase Address (S15). La presenza di questo stato mi permette di gestire sequenze di parole iniziali nulle.

Output Signal	Value	Note
o_inc_add	1	Qualunque sia il risultato del confronto, l'indirizzo di memoria nel Modulo Register Address dovrà comunque essere incrementato, sia per un eventuale scrittura di credibilità, sia nel caso di sequenze con parole iniziali nulle.

Tabella 12: Segnali di Output in S5.

- Write Credibility (S6): Stato in cui viene scritta la credibilità in memoria.

Output Signal	Value	Note
o_inc_add	1	Abilito incremento dell'indirizzo di memoria per una lettura successiva.
o_mem_en	1	Abilito interazione con la memoria.
o_mem_we	1	Abilito scrittura in memoria.
$o_enable_comparator$	1	Abilito il Modulo K-Comparator che verifica se ha terminato di leggere la sequenza di parole.

Tabella 13: Segnali di Output in S6.

- Ask Memory (S7): Stato che controlla l'input i_stop proveniente dal Modulo K-Comparator, il quale indica l'esito del confronto abilitato in S6. Se ho letto i_k parole allora lo stato corrente diventerà Wait (S14). Per quanto riguarda i valori di output sono analoghi a quelli descritti nello stato Ask Memory (S3) 2.7.3. La differenza tra i due consta nel caso in cui i_stop sia uguale a 0, nel quale lo stato corrente sarà Enable Load Value (S8).
- Enable Load Value (S8): Stato analogo a Enable Load Value (S4) descritto in 2.7.3 per quanto riguarda i valori di output. La differenza tra i due consta nello stato prossimo, che in questo caso è Check Result_1 (S9).
- Check Result_1 (S9): Stato in cui viene controllato il valore di i_check , nel caso in cui sia 0 lo stato prossimo sarà Set Correct Output (S11), nel caso opposto sarà invece Reset Credibility (S10). A quanto concerne i valori di output essi sono analoghi a quelli di default presentati in sezione 2.7.2.
- Reset Credibility (S10): Stato in cui viene resettato a 31 il valore della credibilità. Utilizzato per gestire sequenze del tipo '12 0 0 0 14 0', nelle quali a seguito di una parola nulla (non iniziale) è presente una parola non nulla.

Output Signal	Value	Note
o_rst_cred	1	Resetto il valore della credibilità presente nel registro del Modulo Register Credibility a 31.

Tabella 14: Segnali di Output in S10.

- Set Correct Output (S11): Stato per gestire la scrittura del corretto valore in memoria.

Output Signal	Value	Note
o_sel	0	Salvo in Modulo Register Output il valore proveniente da Modulo Register Value, che corrisponde all'ultima parola letta non nulla della sequenza.

Tabella 15: Segnali di Output in S11.

- Write Value (S12): Stato in cui viene scritto l'ultimo valore della sequenza non nullo in memoria.

Output Signal	Value	Note
<i>o_mem_en</i>	1	Abilito la comunicazione con la memoria.
<i>o_mem_we</i>	1	Abilito scrittura di un valore in memoria.
<i>o_dec_cred</i>	1	Decremento il valore di credibilità nel registro del Modulo Register Credibility, poichè se sono arrivato in questo stato significa che ho letto una parola nulla non iniziale nella sequenza e come da specifica dovrò riportare il valore di credibilità diminuito di 1.

Tabella 16: Segnali di Output in S12.

- Increase Address (S13): Stato predisposto all'incremento dell'indirizzo nel Modulo Register Address.

Output Signal	Value	Note
<i>o_inc_add</i>	1	Abilito incremento dell'indirizzo di memoria.

Tabella 17: Segnali di Output in S13.

- Wait (S14): Stato in cui mi trovo quando ho finito di leggere il numero di parole della sequenza. L'introduzione di uno stato di Wait mi permette di poter leggere più sequenze senza dover attendere il segnale di reset del modulo. Al suo interno *o_done* sarà posto a 1 fino a che il segnale di *i_start* non sarà riportato a zero.

Output Signal	Value	Note
<i>o_done</i>	1	Indica che il modulo ha terminato l'analisi della sequenza
<i>o_rst_cred</i>	1	Resetto il valore di credibilità a 31 nel registro del Modulo Register Credibility.
<i>o_rst_counter</i>	1	Resetto il valore del conteggio a 0 nel registro del Modulo Counter, per prepararlo a un nuovo ciclo di conteggio.

Tabella 18: Segnali di Output in S14.

- Increase Address (S15): Stato predisposto all'incremento dell'indirizzo, impiegato per la gestione di sequenze con parole iniziali nulle. I valori di output sono analoghi a quelli dello stato Increase Address (S13) 2.7.3.

3 Risultati sperimentali

3.1 Sintesi

Il componente è stato sintetizzato utilizzando Vivado con una FPGA target Artix-7 xc7a12ticsg325-1L. L'architettura elaborata si sintetizza correttamente e si evidenzia l'assenza di Latch.

Si riporta la tabella estratta dal Report Utilization.

Site Type	Used	Fixed	Available	Util%
LUT as Logic	59	0	8000	0.74
LUT as Memory	0	0	5000	0.00
Register as Flip Flop	75	0	16000	0.47
Register as Latch	0	0	16000	0.00
F7 Muxes	0	0	7300	0.00
F8 Muxes	0	0	3650	0.00

Tabella 19: Utilization report.

Si riporta la tabella estratta dal Report Timing.

Slack (MET)	16.551ns
Requirement	20.000ns
Data Path Delay	3.313ns

Tabella 20: Timing report.

Il valore di Slack (required time - arrival time) riportato permette di concludere che la rete ottiene ottimi risultati in termini di timing. A fronte della presenza di uno Slack elevato, il sistema è meno suscettibile a errori causati da ritardi imprevisti, ciò contribuisce a una maggiore affidabilità del sistema e alla possibilità di implementare funzionalità aggiuntive nell'architettura senza compromettere le prestazioni temporali.

3.2 Simulazioni

Al fine di verificare il funzionamento del modulo, oltre al test fornito, sono stati sviluppati testbench che simulassero casi di sequenze particolari e presenza di segnali di inizio elaborazione in concomitanza con segnali di reset. I test sono stati verificati sia su simulazione Behavioral che sulla rete sintetizzata in Post-Synthesis Functional.

I casi valutati (e correttamente superati) sono i seguenti:

- Sequenza di parole nulle.
- Valore di i_k nullo.
- Sequenza di parole che portasse il valore di credibilità al suo valore limite zero.
- Sequenza di parole inizialmente nulle e successivamente diverse da zero.
- Presenza di più reset asincroni.
- i_rst e i_start uguali a 1 contemporaneamente.

4 Conclusioni

A valle dei risultati positivi ottenuti da simulazioni eseguite sull'architettura progettata si può concludere che ci sia stato un rispetto pieno dei requisiti di specifica. In merito a possibili applicazioni esterne, la nostra architettura potrebbe essere utilizzata per controllare la correttezza di determinate sequenze in memoria, infatti avendo implementato un Modulo 2.1 che paragonasse una parola di memoria con il valore zero, questo può essere esteso con una banale modifica a casi di controllo maggiormente specifici. In alternativa modificando la finite state machine con l'aggiunta di un ulteriore stato, si può controllare che tutti i valori di una determinata sequenza di memoria (e non solo quelli in posizioni pari rispetto all'indirizzo di partenza) soddisfino determinati parametri.